

IT5002 Computer Systems and Applications

2024/25 Semester II

Final Assessment

SOLUTIONS

Question 1. (9 MARKS)

Consider the following program, where the base address of a 100 element array is in \$s0.

In all parts we assume that this program is executed to completion.

```
        addi $t0, $s0, 400
        add $s1, $zero, 0
L:      lw $t1, 0($s0)
        add $s1, $s1, $t1
        addi $s0, $s0, 4
        slt $t1, $s0, $t0
        bne $t1, $zero, L
```

- a. How many instructions are executed by this program? (2 marks)

Outside the loop: 2 instructions

Inside the loop: $100 \times 5 = 500$ instructions

Total = 502 instructions

- b. **Assuming that your answer to part a. is 637 instructions**, how long does it take in microseconds to execute this code on a single-cycle CPU with a clock rate of 100 MHz? Express your answer to two decimal places if it is not a whole number. (1 microsecond = 10^{-6} seconds). (2 marks)

100 MHz clock means that each clock cycle will take 0.01 microseconds

$502 \times 0.01 = 6.37$ microseconds.

- c. **Again, assuming that your answer to part a. is 637 instructions**, How long does it take to execute this program on a CPU with an ideal **4-stage** pipeline? Assume the following timings for each stage (1 ns = 10^{-9} seconds)

Fetch	Decode	Execute	Writeback
10 ns	5 ns	5 ns	10 ns

Ignore dependencies between instructions and branching hazards (a branch hazard is caused by the pipeline not knowing whether to fetch the next instruction or the instruction from the branch target).

State your answer in ns, in two decimal places if your answer is not a whole number. (1 ns = 10^{-9} seconds) (2 marks)

of cycles = 4 + 636 = 640 cycles. Each stage will have a 10ns time. Total time = 6400 ns.

- d. Assuming a multicycle CPU where the stages take the following time in ns (1 ns = 10^{-9} seconds)

Fetch	Decode	Execute	Memory	Writeback
15 ns	5 ns	10 ns	15 ns	5 ns

Instructions take the following number of clock cycles each:

Instruction Type	Number of Clock Cycles
Arithmetic	4
Load/Store	5
Branching	3

If our program executed a total of 350 arithmetic, 180 load/store and 95 branching instructions, how long would it take to execute it?

Express your answer in ns, in two decimal places if it is not a whole number. (1 ns = 10^{-9} seconds) (3 marks)

of ns for each stage = 15 ns

of cycles = $350 \times 4 + 180 \times 5 + 95 \times 3 = 2585$ cycles

Time taken = $2585 \times 15 = 38775$ ns

Question 2. (16 MARKS)

We continue again with the same program from Question 1:

```
        addi $t0, $s0, 400      # i1
        add $s1, $zero, 0      # i2
L:      lw $t1, 0($s0)         # i3
        add $s1, $s1, $t1      # i4
        addi $s0, $s0, 4       # i5
        slt $t1, $s0, $t0      # i6
        bne $t1, $zero, L      # i7
```

All instructions and word sizes are 32-bits.

We consider first the instruction cache, which is a two-way set associative cache totalling 64 bytes, with a cache block size of 2 words.

- a. How many sets would our instruction cache have? (2 marks)

Each block = 2 words = 8 bytes

Each set = 2 blocks = 16 bytes

of sets = $64 / 16 = 4$ sets

- b. If addresses are 32-bits long, how long in bits is the tag portion of the address? (2 marks)

Offset portion = $\log_2 8 = 3$ bits

Set index portion = $\log_2 2$ bits

Tag portion = $32 - 3 - 2 = 27$ bits

The data cache is a direct-mapped cache totaling 128 bytes with a cache block size of 4 words:

- c. How many blocks would our **data** cache have? (2 marks)

Total # of bytes per block = $4 \times 4 = 16$ bytes

Total number of blocks = $\frac{128}{16} = 8$ blocks

- d. If addresses are 32-bits long, how long in bits is the tag portion of the address? (2 marks)

Offset = $\log_2 16 = 4$ bits

Block index = $\log_2 8 = 3 \text{ bits}$

Tag = $32 - 4 - 3 = 25 \text{ bits}$

- e. Assuming that the address of the first instruction (Marked “# i1”) is at 0x2F1C4C, what is the hit-rate of our cache for the program given above after it has finished running? State your answer in percent (e.g. 15%), to two decimal places if the answer is not a whole number. (4 marks)

i0	0x2F1C48	0010 1111 0001 1100 010 01 000
i1	0x2F1C4C	0010 1111 0001 1100 010 01 100
i2	0x2F1C50	0010 1111 0001 1100 010 10 000
i3	0x2F1C54	0010 1111 0001 1100 010 10 100
i4	0x2F1C58	0010 1111 0001 1100 010 11 000
i5	0x2F1C5C	0010 1111 0001 1100 010 11 100
i7	0x2F1C60	0010 1111 0001 1100 011 00 000

0x2F1C4C = 0010 1111 0001 1100 010 01 100

Set	Block 0	Block 1
0	i6i7	
1	i0i1	
2	i2i3	
3	i4i5	

In the first iteration, there are 7 accesses with 4 cache misses. This works out to 1 iteration. For remaining 99 iterations there are 495 accesses with 0 misses. Total = 502 accesses with 4 misses. Hit rate = $1 - \frac{4}{502} = 0.99 = 99\%$

- f. Assuming that register \$s0 contains the address 0x1CFB2A40, what is the hit rate of our data cache in percent? Express your answer to two decimal places if it is not a whole number. (4 marks)

0x1CFB2A40 = 0001 1100 1111 1011 0010 1010 0 100 0000

So the first byte is loaded into word 0 of block 4. Since we access every word exactly once and each load loads 4 words, we have a miss and 3 hits each time, giving us 75% hit rate.

Question 3. (12 MARKS)

We consider 3 processes in a fixed-priority scheduling system with a single CPU but two independent I/O subsystems. **Reminder: There are no pre-emptions for I/O subsystems. Additionally, if two processes want to start using the same piece of I/O at the same time, the higher priority process gets to go first.**

The processes have the following priority, start time and execution profiles:

Process	Priority	Start Time	Execution Profile
P0	Low	2	$C3 - IO_22 - C2 - IO_12 - C1$
P1	High	1	$C2 - IO_12 - C2 - IO_11 - C2$
P2	Medium	0	$C3 - IO_21 - C2 - IO_11 - C1$

Here Cn means the process uses the CPU for n cycles, IO_{xy} means the process uses IO unit x for y cycles. The “Start Time” of the process is not the time the process starts running on the CPU, but rather the time the process is first added to the ready queue.

Time	0	1	2	3	4	5	6	7	8	9	10	11
CPU	P2	P1	P1	P2	P2	P1	P1	P2	P1	P1	P2	P0
IO1				P1	P1			P1				P2
IO2						P2						
Ready		P2	P2, P0	P0	P0	P0	P2, P0	P0	P2, P0	P2, P0	P0	
Time	12	13	14	15	16	17	18	19	20	21		
CPU	P2	P0	P0			P0	P0		P0			
IO1								P0	P0			
IO2				P0	P0							
Ready	P0											

- a. What is the response time of each process? (This is the time the process waits until the first time it gets to execute) (3 marks)

P0: 9 cycles

P1: 0 cycles

P2: 0 cycles

- b. What is the turn-around-time for each process? The turn-around-time is the time between the process's start time and the time it ends. (3 marks)

P0: $22 - 2 = 20$ cycles

P1: $10 - 1 = 9$ cycles

P2: $13 - 0 = 13$ cycles

- c. What is the total waiting time of each process? (This is the total time the process spends waiting for the CPU) (3 marks)

P0: $20 - 10 = 10$ cycles

P1: $9 - 9 = 0$ cycles

P2: $13 - 8 = 5$ cycles

- d. What is the CPU utilization of the 3 processes in percent (e.g. 10%)? State your answer in two decimal places if it is not a whole number. (3 marks)

$$U = 1 - \frac{4}{22} = 0.8182 = 81.82\%$$

Question 4. (5 MARKS)

We have the following three concurrent processes sharing semaphores binary S1, S2 and S3:

Shared Semaphores with Initial Values		
Semaphore S1 = 0; Semaphore S2 = 1 Semaphore S3 = 0		
Process P1	Process P2	Process P3
P1-1: up(S1); P1-2: down(S1); P1-3: up(S2); P1-4: up(S1);	P2-1: down(S1); P2-2: up(S3); P2-3: up(S1); P2-4: down(S1); P2-5: up(S3);	while(1) { P3-1: down(S2); P3-2: printf("Hello world!\n"); P3-3: down(S3); }

(Note: P1-1, P2-1, etc are just statement labels and can be ignored)

What is the maximum number of times will the phrase "Hello world!" appear when the three processes are run concurrently? Assume that semaphores unblock processes in first-in-first-out (FIFO) order. I.e. if P1 blocks on S, then P2 blocks on S, and P3 does up(S), P1 will always be unblocked first. (4 marks)

P2-1: P2 downs on S1 so will block

P3-1: P3 downs on S2, decrements it to 0

P3-2: >P3 prints Hello World

P3-3: It downs on S3 and blocks

P1-1: P1 ups S1

P2-1: P2 unblocks. P1-2: P1 blocks on S1

P2-2: P2 ups S3

P3-3: P3 will unblock on S3, loop around and block on S2 at P3-1

P2-3: P2 ups S1 and unblocks P1 at P1-2

P2-4: P2 then blocks on S1

P1-3: P1 ups S2, unblocks P3 at P3-1

P3-2: >P3 prints Hello world

P3-3 P3 blocks on S3

P1-4: P1 ups S1 and exits, unblocks P2 at P2-4

P2-5: P2 ups S3 and exits, unblocking P3 at P3-3

P3-1: P3 blocks on S2

Answer: Twice

Question 5. (8 MARKS)

We consider an inode based file system with the following specifications:

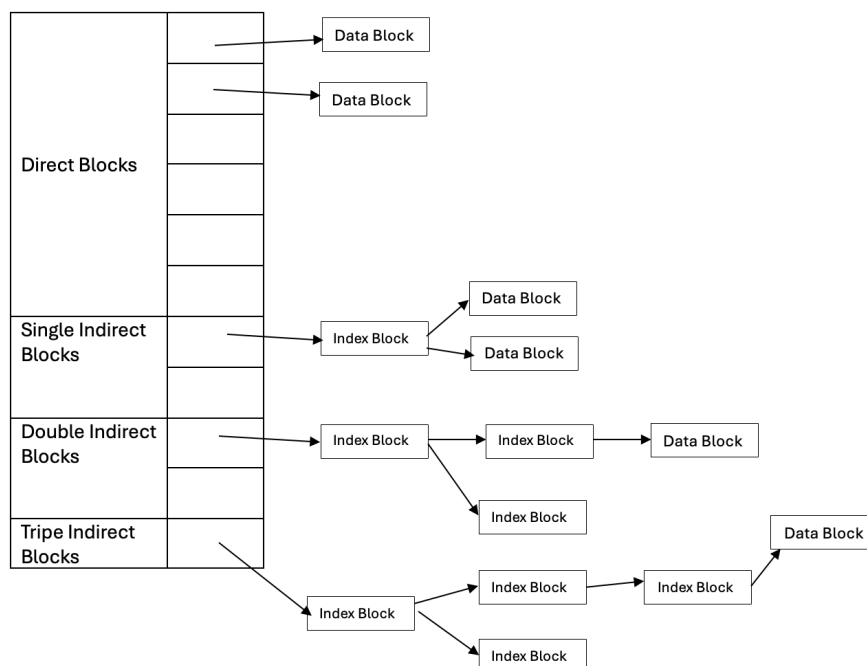
of direct blocks: 6

of single-indirect blocks 2

of double-indirect blocks 2

of triple-indirect blocks 1

This is partially shown below:



All block pointers are 8 bytes long, and data blocks are 4096 bytes long.

- a. How many block pointers can each data block hold? (4 marks)

One block is 4096 bytes long, each 8 bytes, so # of pointers = 512

- b. Assuming that the answer to part a is 256 file pointers (blocks are still 4096 bytes long), what is the maximum size of a file in this file system? Express your answers in gigabytes (1 gigabyte = 2^{30} bytes) to two decimal places if it is not a whole number. (4 marks)

of bytes in direct blocks = $6 \times 4096 = 24576 \text{ bytes}$

of bytes in single indirect blocks = $2 \times (256 \times 4096) = 2097152 \text{ bytes}$

of bytes in double indirect blocks = $2 \times (256 \times 256 \times 4096) = 536870912 \text{ bytes}$

of bytes in triple indirect blocks = $1 \times (256 \times 256 \times 256 \times 4096) = 68719476736$

Total = 69258469376 bytes

= 64.51 gigabytes